1_LangChain_Foundation

NAME: ANANYAA SREE SP

SRN: PES2UG23CS066

SECTION: A

With temp=2 added

```
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0)

# Model C:
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=2.0)
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental image formed in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.
```

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or plan that exists in the mind.
Run 2: An idea is a thought or concept that arises in the mind, serving as a suggestion, plan, or basis for understanding and action.
```

```
print("---  (Temp=2) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
---  (Temp=2) ---
Run 1: An idea is a thought, concept, or plan conceived in the mind.
Run 2: An idea is a mental conception, thought, or notion formed in the mind, often as a plan, suggestion, or understanding of something.
```

Observation

- **Temp 0.0:** Both runs produced identical definitions.
- **Temp 1.0:** Both runs produced similar but differently phrased definitions.
- **Temp 2.0:** Both runs produced more varied sentence structures, but the core meaning remained consistent.

LangChain and Temperature Analysis

Observations

1. LangChain provides a unified interface to interact with LLMs.
2. Models process input as tokens, and token usage affects cost and context limits.
3. With **temperature = 0.0**, repeated prompts produced exactly the same output.
4. With **temperature = 1.0**, repeated prompts produced slightly varied but meaningful outputs.
5. With **temperature = 2.0**, outputs showed higher variation in wording, but remained logically similar.
6. At temperature 2.0, creativity increased but factual meaning was still preserved.

Inferences

- LangChain abstracts API-level complexity and allows easy switching between models. Tokens act as the computational units of LLMs and directly impact pricing and memory.
- Temperature controls randomness, not intelligence. Low temperature is suitable for factual, mathematical, and deterministic tasks. High temperature is suitable for creative and generative tasks.
- Temperature amplifies the sampling behavior of a model. Strong models degrade gracefully, while weaker models become unstable when randomness increases.

Prompts and Parsers

Observations

1. System messages influence the tone and behavior of the model.
2. The same question produces different responses depending on system instructions.
3. Large inputs increase token consumption and may exceed context limits.
4. Prompt templates structure input in a standardized manner.
5. Model outputs are returned as structured objects, not plain strings.
6. Output parsers extract clean text from model responses.

Inference

Model behavior is highly sensitive to how input is structured. Proper role assignment, controlled prompt design, and standardized templates directly affect output reliability and efficiency within context limits. Separating prompt formatting and output parsing improves modularity, maintainability, and application-level control without altering the model itself.

LCEL (LangChain Expression Language)

Observations

1. Manual chaining requires invoking each step separately.
2. LCEL allows combining multiple components into a single pipeline.
3. The chain can be modified easily by replacing or adding components.
4. The model can retrieve factual information and perform simple reasoning tasks.

Inference

LCEL enables structured pipeline design where components can be composed, extended, or replaced without restructuring the entire workflow. This promotes scalable and maintainable AI application development. However, since LLM reasoning is probabilistic, so output may not always be fully reliable.

2_Prompt_Engineering

Part 2a: The Anatomy of a Prompt

Observations

1. AI outputs vary due to stochastic token prediction; randomness depends on temperature.
2. Low temperature (0.0) produces consistent, deterministic outputs.
3. High temperature increases variability and creativity in responses.
4. Structured prompts (CO-STAR framework) guide context, objective, style, tone, audience, and output format.
5. Ambiguity in prompts leads the model to fill gaps with probability-based guesses.

Inferences

- Temperature controls randomness, not intelligence; low values are suitable for factual tasks, high for creative tasks.
- Structured prompts reduce hallucination and improve output reliability.
- Providing explicit context and constraints ensures the model follows desired instructions.

Part 2b: Zero-Shot to Few-Shot

Observations

1. Zero-shot relies solely on pre-trained knowledge without examples.
2. Few-shot provides input-output examples, allowing the model to detect patterns.
3. Attention mechanism uses prior examples to influence predictions.
4. Poor-quality examples lead the model to replicate incorrect patterns.

Inferences

- In-context learning allows LLMs to adapt behavior without retraining.
- Example quality directly impacts output reliability and correctness.
- Few-shot prompting leverages attention for pattern matching, improving reasoning over zero-shot.

Part 2c: Advanced Templates and Theory

Observations

1. Hard prompts modify input text; soft prompts modify model weights internally.
2. Hard prompts are cheaper and fast; soft prompts are data- and compute-intensive.
3. Dynamic few-shot selects the most relevant examples from a large database.
4. Separating instructions from examples improves model focus and output quality.

Inferences

- Hard prompts are ideal for rapid prototyping; soft prompts for domain-specific adaptation.
- Dynamic few-shot optimizes context window usage, improving in-context learning efficiency.
- Clear separation between instructions and examples reduces model confusion and enhances attention effectiveness.

3_Advanced_Prompting

1. Chain of Thought (CoT)

Observation:

- Standard prompt often gave wrong answers.
- Adding let's think step by step led to intermediate steps and correct results.

Inference:

- Step-by-step guidance improves logical reasoning in LLMs.
- Writing out steps acts like "thinking aloud" for the model.

2. Tree of Thoughts (ToT)

Observation:

- Model generated multiple solutions to a problem.
- A judgeprompt could select the most practical/sustainable solution.

Inference:

- Exploring multiple paths improves decision quality.
- LLMs can evaluate alternatives when guided properly.

3. Graph of Thoughts (GoT)

Observation:

- Model produced independent ideas (plots) in different genres.
- Aggregating them gave a richer, creative final output.

Inference:

- Combining outputs enhances creativity and synthesis.
- GoT is useful for complex, multi-domain problem solving.

Summary

- Smaller models need structured prompts to perform well.
- CoT gives better logic and stepwise reasoning.
- ToT gives better decisions via multiple options.
- GoT helps in creative synthesis across multiple ideas.
- Trade-off: higher reasoning quality comes with increased cost/latency.Structured prompting turns LLMs from simple text generators into reasoning systems.

4_RAG_and_Vector_Stores

Embeddings and Vector Space

Observations

1. Computers cannot understand text natively, words are just byte sequences.
2. Embeddings translate words into vectors, where similar words have nearby vectors.
3. The all-MiniLM-L6-v2 model produces 384-dimensional vectors for English text.
4. Cosine similarity measures closeness of vectors: 1.0 = identical, 0.0 = unrelated, -1.0 = opposite.
5. "Cat" and "Dog" vectors have higher similarity than "Cat" and "Car", reflecting semantic meaning.

Inferences

- Embeddings allow machines to reason about semantic similarity numerically.
- High-dimensional vectors encode multiple latent features (e.g., category, color, context).
- Cosine similarity provides a robust measure for comparing meanings in vector space.
- Modern search and RAG systems rely fundamentally on this vector representation of knowledge.

Naive RAG Pipeline

Observations

1. LLMs can hallucinate when relying solely on parametric memory.
2. RAG introduces non-parametric memory by retrieving relevant external documents.
3. The pipeline involves retrieval → augmentation → generation.
4. FAISS efficiently stores embeddings and retrieves nearest neighbors.
5. Using a knowledge base (documents) allows LLMs to answer questions with grounded facts.

Inferences

- RAG improves factual accuracy of LLM outputs by providing explicit context.
- Retrieval can be abstracted via vector stores, making it transparent to the LLM.
- Combining retriever results with prompts allows the LLM to focus on reasoning rather than recall.
- Proper indexing and embedding quality directly influence RAG performance.

Indexing Algorithms (FAISS)

Observations

1. Flat index (brute force) guarantees 100% accuracy but is slow for large datasets.
2. IVF partitions vectors into clusters to reduce search space, trading some accuracy for speed.
3. HNSW builds a graph of vectors with hierarchical connections for fast retrieval.
4. PQ compresses vectors to reduce memory usage, sacrificing precision.
5. Each indexing method offers a trade-off between speed, accuracy, and memory usage.

Inferences

- Choice of indexing algorithm depends on dataset size, hardware constraints, and retrieval needs.
- Flat index is ideal for small datasets; IVF and HNSW scale better for millions of vectors.
- PQ is essential when RAM is limited or billions of vectors must be stored.
- Understanding algorithmic trade-offs allows designing efficient vector search systems for RAG and other AI applications.